

Betriebssysteme

Kapitel 5: Filesysteme

Prof. Dr. Raphael Herding

Inhalte des Kapitels (1)

5. Filesysteme

- Einleitung
 - Aufbau von Festplatten
 - Bestandteile von Filesystemen
 - Operationen und Attribute
- Beispiel: UNIX/Linux
 - Baumstruktur, Pfade
 - *Links*
 - *Mounten* von Filesystemen
 - *Inodes*
 - UNIX-Filesysteme: UFS, BSD 4.2, EXT2
- ...

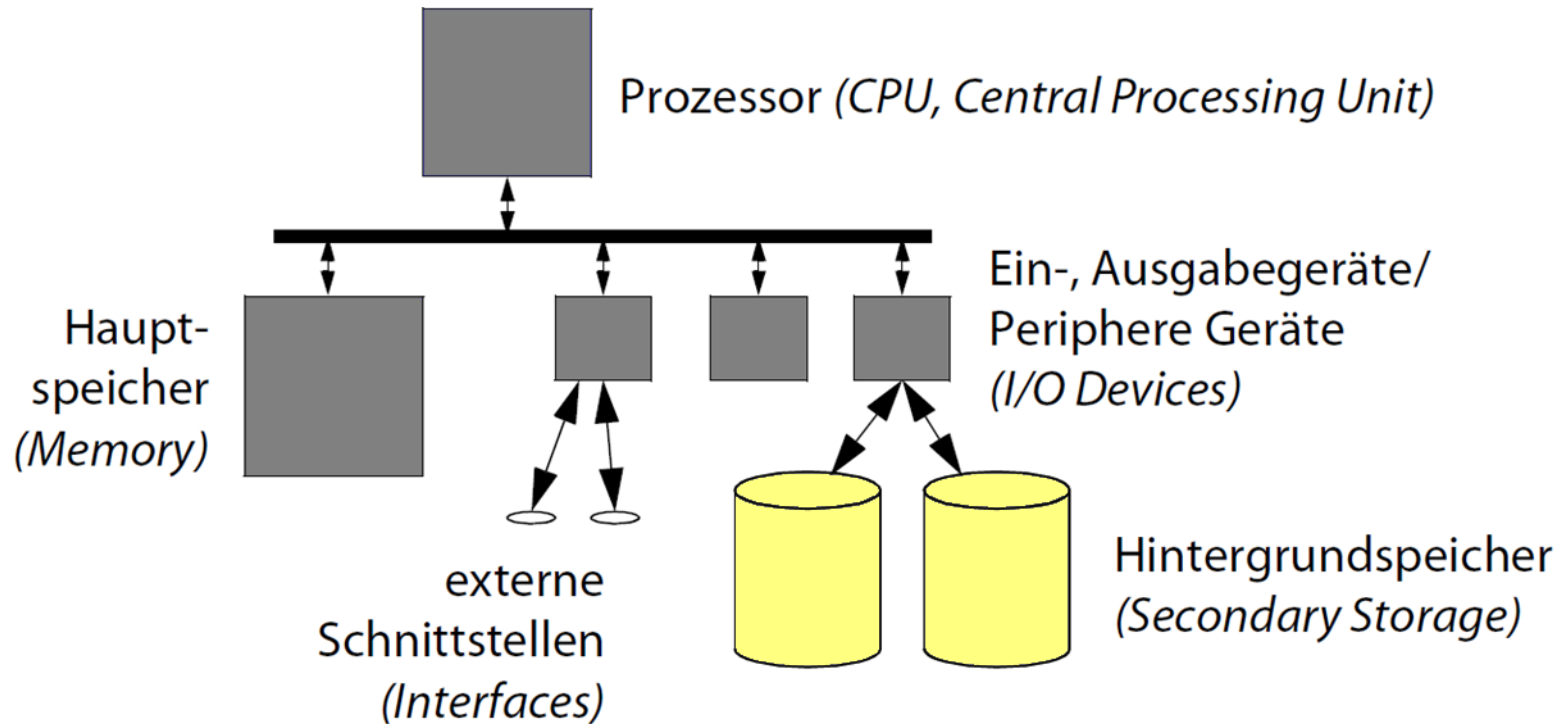
Inhalte des Kapitels (2)

5. Filesysteme

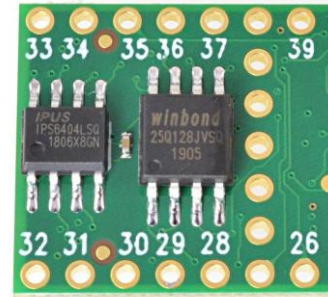
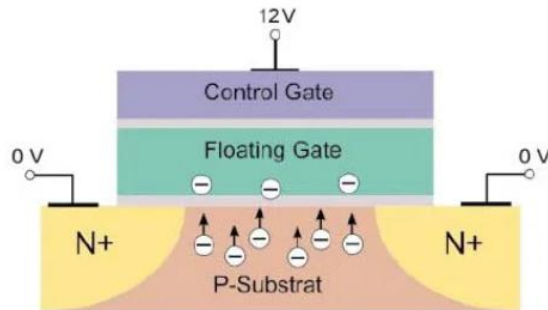
- ...
- Zuverlässige Filesysteme
 - Journal (*Log*); *Undo & Redo*
 - *Log-Structured File Systems*
- Limitierung der Plattennutzung, fehlerhafte Blöcke

Einordnung

Betroffene physikalische Ressourcen



Schematischer Aufbau einer Solid State Disc (SSD)



- Schema einer Flash-Speicherstelle
- Das Floating-Gate erzeugt einen Käfig, der bei angelegter Spannung Elektronen einfangen kann und dann permanent hält – auch ohne Spannung.
- Die gefangenen Elektronen im Floating Gate beeinflussen das elektrische Feld des anliegenden Control Gates und damit die Leitfähigkeit der Flash-Zelle.
- Die messbare Schwellenspannung steigt mit der Ladung des Floating Gates (hoch = geladen = 1 oder niedrig = ungeladen = 0).
- Durch Löschspannung können Elektronen wieder aus Gate „herausgedrückt“ werden (ungeladen = 0).
- Ein Flash-Speicherchip fasst mehrere Milliarden Zellen zusammen.

Motivation (1)

Filesysteme speichern Daten und Programme persistent in Dateien

- Betriebssystemabstraktion zur Nutzung von Hintergrundspeichern (z.B. Festplatten, Flash-Speicher, DVDs, Bandlaufwerke, ...)
 - Benutzer muss sich nicht um die Ansteuerung verschiedener Speichermedien kümmern
 - Einheitliche Sicht auf den Sekundärspeicher

Filesysteme bestehen aus

- Dateien (*files*)
- Verzeichnissen (*directories*)
- Partitionen (*partitions*)

Motivation (2)

Datei

- Speichert Daten oder Programme

Verzeichnis

- Fasst Dateien (und Verzeichnisse) zusammen

Partitionen

- Eine Menge von Verzeichnissen und deren Dateien
- Dient zum physischen oder logischen Trennen von Dateimengen
 - *physisch*: Festplatte, Diskette
 - *logisch*: Teilbereich auf Festplatte oder CD, Zusammenfassung mehrerer Teilbereiche

Dateien (1)

Kleinste Einheit, in der etwas auf den Hintergrundspeicher geschrieben werden kann.

Typische Attribute

- *Name* – Symbolischer Name, vom Benutzer les- und interpretierbar
 - z.B. **AUTOEXEC.BAT**
- *Typ* – Für Dateisysteme, die verschiedene Dateitypen unterscheiden
 - z.B. sequentielle Datei, zeichenorientierte Datei
- *Ortsinformation* – Wo werden die Daten physisch gespeichert?
 - Gerätenummer, Nummern der Plattenblöcke
- *Größe* – Länge der Datei in Größeneinheiten (z.B. Bytes, Blöcke)
 - Steht in engem Zusammenhang mit der Ortsinformation
 - Wird zum Prüfen der Dateigrenzen z.B. beim Lesen benötigt

Dateien (2)

Typische Attribute (fortges.)

- Zeitstempel – z.B. Zeit und Datum der Erstellung, letzten Änderung
 - Unterstützt Backup, Entwicklungswerkzeuge, Benutzerüberwachung
- Rechte – Zugriffsrechte, z.B. Lese-, Schreibberechtigung
 - z.B. nur für den Eigentümer schreibbar, für alle anderen nur lesbar
- Eigentümer – Identifikation des Eigentümers
 - Eventuell eng mit den Rechten verknüpft
 - Zuordnung beim *Accounting* (Abrechnung des Plattenplatzes)

Verzeichnisse (1)

Ein Verzeichnis gruppiert Dateien und evtl. andere Verzeichnisse

Gruppierungsalternativen:

- Verknüpfung mit der Benennung
 - Verzeichnis enthält Namen und Verweise auf Dateien und andere Verzeichnisse, z.B. UNIX, MS-DOS
- Gruppierung über Bedingung
 - Verzeichnis enthält Namen und Verweise auf Dateien, die einer bestimmten Bedingung gehorchen (z.B. gleiche Gruppennummer in CP/M, eigenschaftsorientierte und dynamische Gruppierung in BeOS-BFS)

Verzeichnis ermöglicht das Auffinden von Dateien

- Vermittlung zwischen externer/interner Bezeichnung (Filename – Blöcke)

Roter Faden

5. Filesysteme

- Einleitung
 - Aufbau von Festplatten
 - Bestandteile von Filesystemen
 - Operationen und Attribute
- Beispiel: UNIX/Linux
- Beispiel: FAT32
- Beispiel: NTFS
- Zuverlässige Filesysteme
- Limitierung der Plattennutzung, fehlerhafte Blöcke

Beispiel: UNIX/Linux

Datei

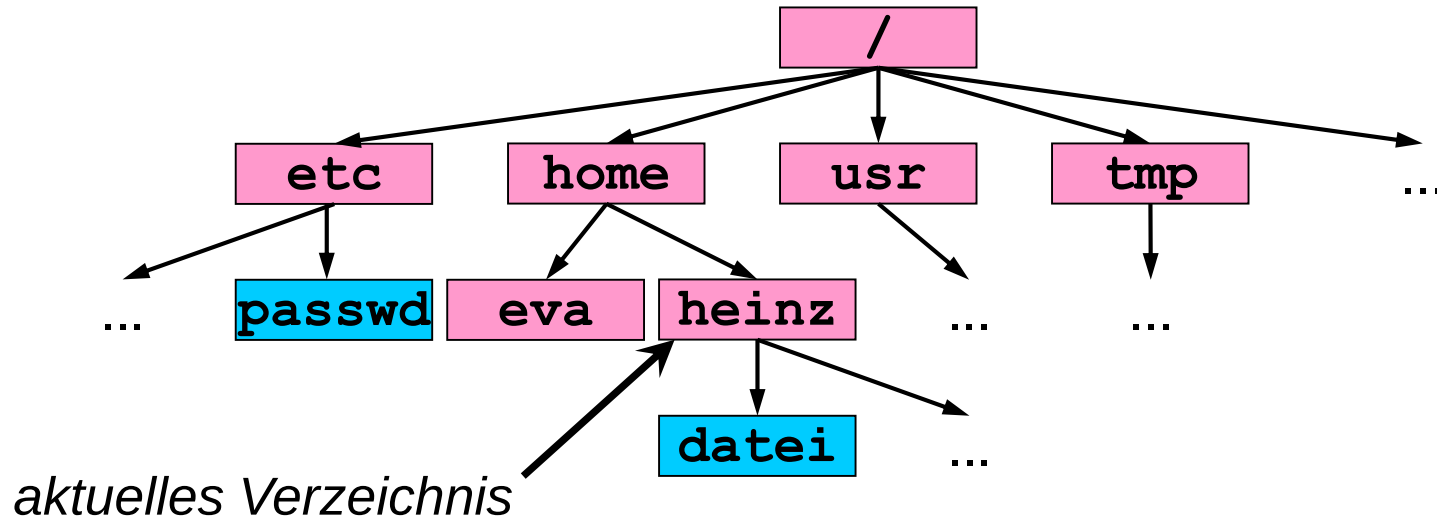
- Einfache, unstrukturierte Folge von Bytes
- Beliebiger Inhalt; für das Betriebssystem ist der Inhalt transparent
- Dynamisch erweiterbar
- Zugriffsrechte: lesbar, schreibbar, ausführbar

Verzeichnis

- Baumförmig strukturiert
 - Knoten des Baums sind Verzeichnisse
 - Blätter des Baums sind Verweise auf Dateien (*Links*)
- Jedem UNIX-Prozess ist zu jeder Zeit ein aktuelles Verzeichnis (*Current Working Directory*) zugeordnet
- Zugriffsrechte: lesbar, schreibbar, durchsuchbar, „nur“ erweiterbar

Pfadnamen (1)

Baumstruktur

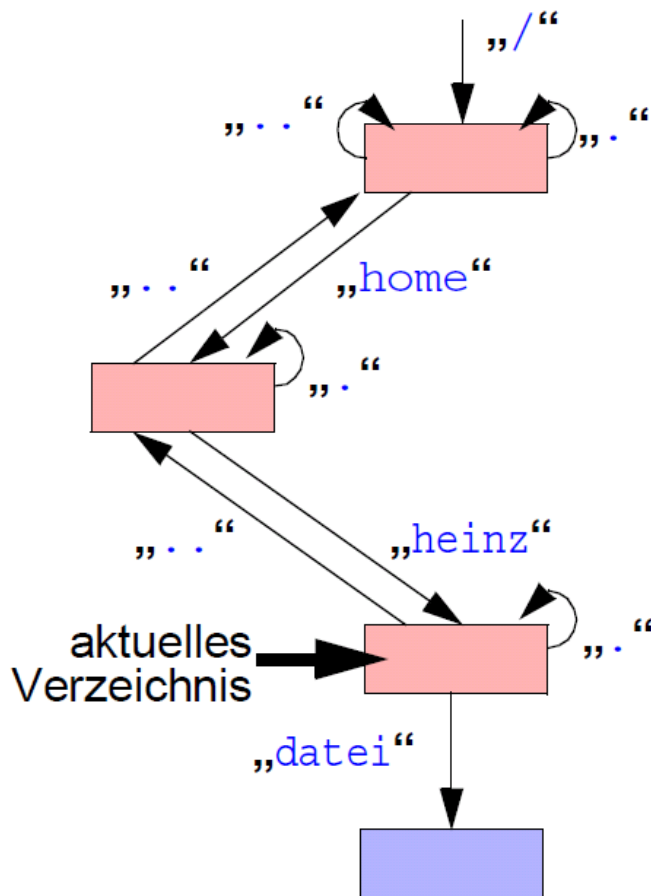


Pfade

- z.B. `/home/heinz/datei`, `/tmp`, `datei`
- „/“ ist Trennsymbol (*Slash*); führender „/“ bezeichnet Wurzelverzeichnis; sonst Beginn implizit mit dem aktuellen Verzeichnis

Pfadnamen (2)

Eigentliche Baumstruktur

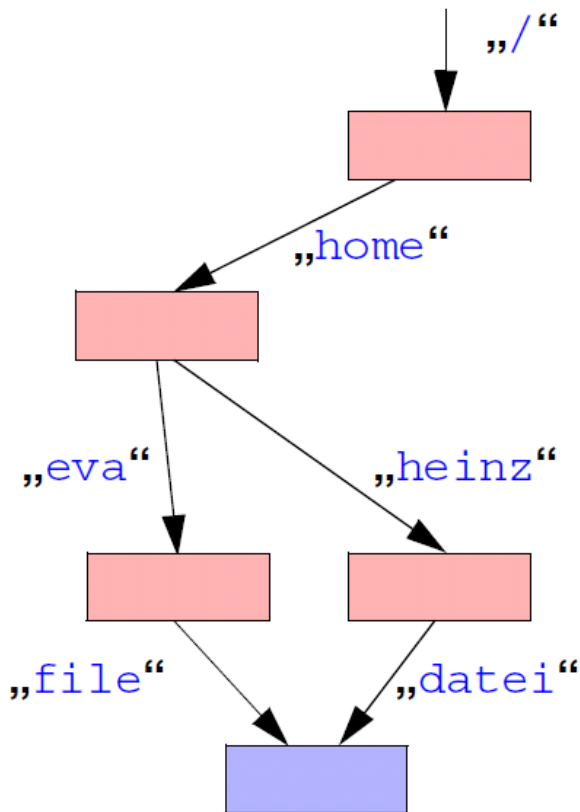


- Benannt sind nicht Dateien und Verzeichnisse, sondern die Verbindungen zwischen ihnen
 - Verzeichnisse und Dateien können auf verschiedenen Pfaden erreichbar sein, z.B. `../heinz/datei` und `/home/heinz/datei`
- Jedes Verzeichnis enthält einen Verweis auf sich selbst (`..`) und einen Verweis auf das darüber liegende Verzeichnis im Baum (`.`)

Pfadnamen (3)

Links (Hard Links)

- Dateien können mehrere auf sich zeigende Verweise besitzen, sogenannte *Hard Links* (nicht jedoch Verzeichnisse)



- Die Datei hat zwei Einträge in verschiedenen Verzeichnissen, die völlig gleichwertig sind:
`/home/eva/file`
`/home/heinz/datei`
- Datei wird erst gelöscht, wenn letzter *Link* gekappt wird.

Eigentümer und Rechte

Eigentümer

- Benutzer werden durch eindeutige Nummer (*User-ID*, *UID*) repräsentiert
- Ein Benutzer kann einer oder mehreren Benutzergruppen angehören, die durch eine eindeutige Nummer (*Group-ID*, *GID*) repräsentiert werden
- Dateien/Verzeichnisse sind genau einem Benutzer/einer Gruppe zugeordnet

Rechte auf Dateien

- Lesen, Schreiben, Ausführen (nur vom Eigentümer veränderbar)
- Einzeln für den Eigentümer, für Angehörige der Gruppe und für alle anderen einstellbar

Rechte auf Verzeichnissen

- Lesen, Schreiben (Löschen/Anlegen von Dateien etc.), Durchgangsrecht

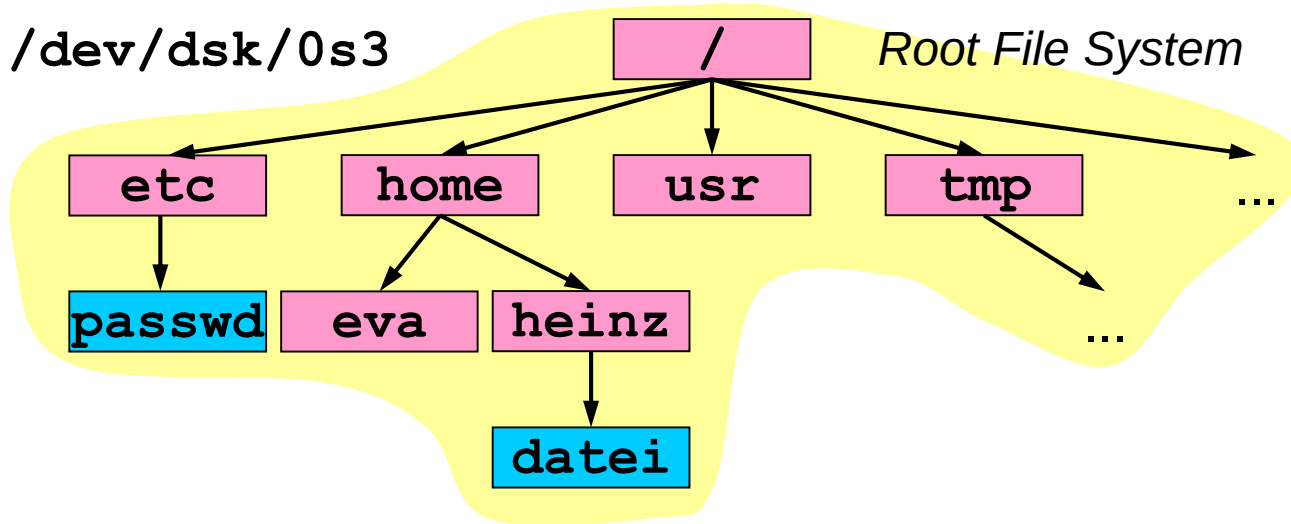
Montieren/Mounten des Dateibaums (1)

Der UNIX-Dateibaum kann aus mehreren Partitionen zusammenmontiert werden

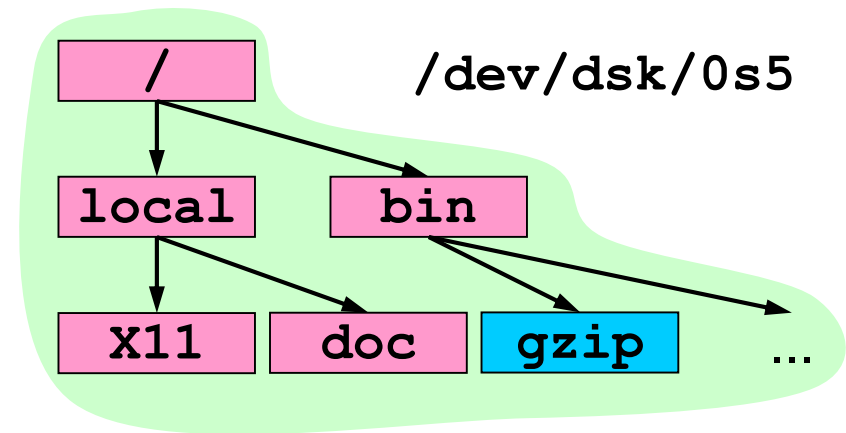
- Partition wird Dateisystem genannt (*File System*)
- Wird durch blockorientierte Spezialdatei repräsentiert (z.B. `/dev/dsk/0s3`)
- Das Montieren wird *Mounten* genannt
- Ausgezeichnetes Dateisystem ist das *Root File System*, dessen Wurzelverzeichnis gleichzeitig Wurzelverzeichnis des Gesamtsystems ist
- Andere Filesysteme können mit dem Befehl **mount** in das bestehende System hineinmontiert werden

Montieren/Mounten des Dateibaums (2)

Beispiel

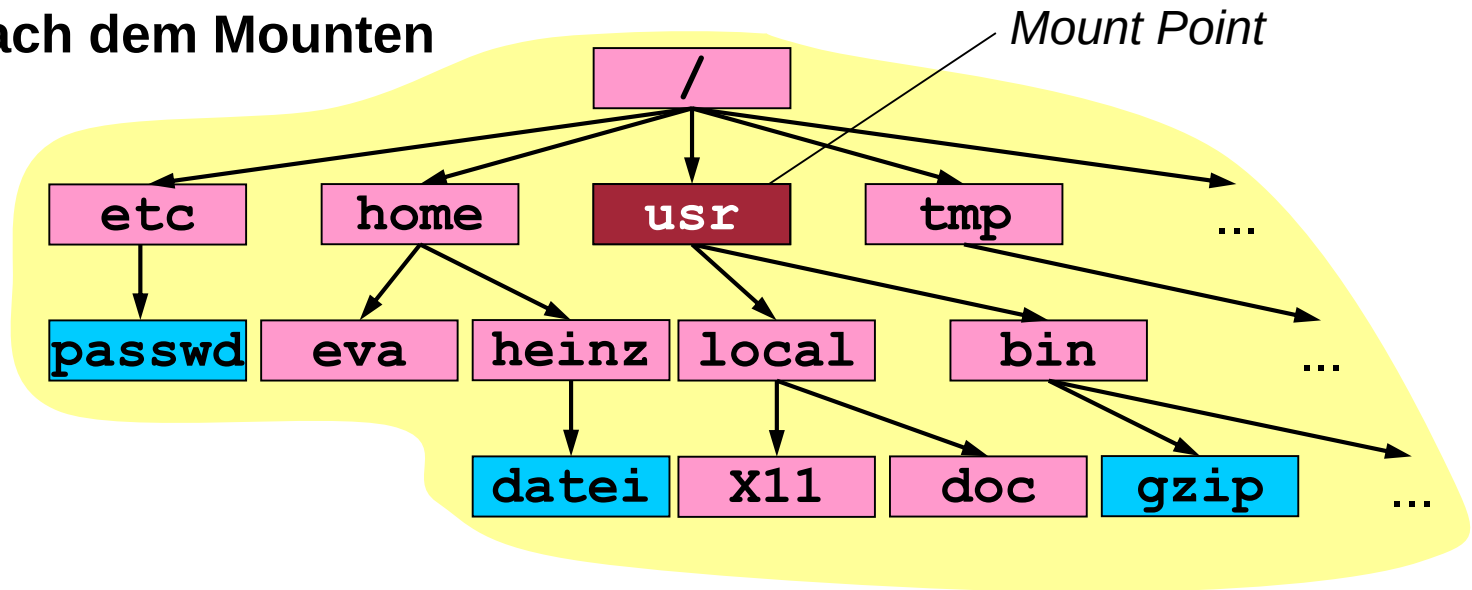


`mount /dev/dsk/0s5 /usr`



Montieren/Mounten des Dateibaums (3)

Beispiel nach dem Mounten

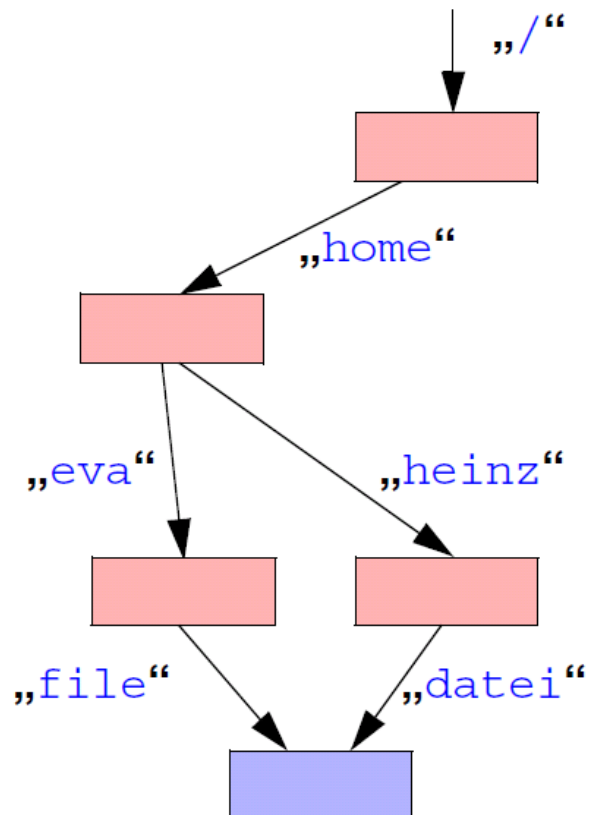


```
mount /dev/dsk/0s5 /usr
```

Inodes (1)

Attribute einer Datei und Ortsinformationen über ihren Inhalt werden in einem sogenannten *Inode* gehalten (ein Block auf Platte)

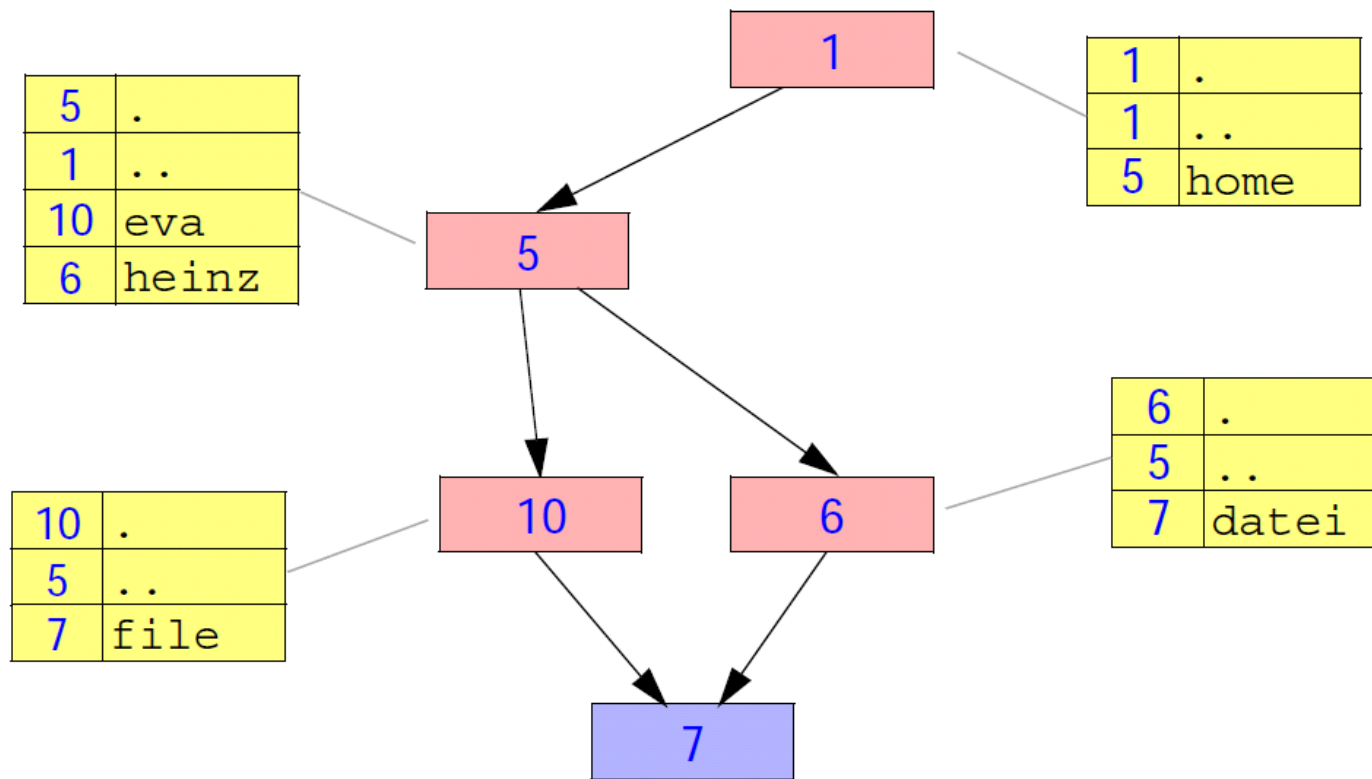
- *Inodes* pro Partition nummeriert (*Inode Number*)



logischer Dateibaum

Inodes (2)

Verzeichnisse enthalten lediglich Paare von Namen und *Inode*-Nummern



tatsächlich gespeicherter Baum

Inodes (3)

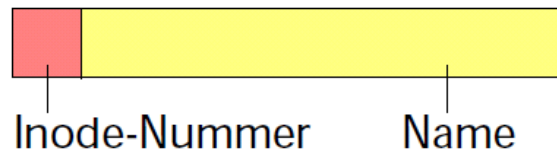
Inhalt eines *Inodes*

- *Inode*-Nummer
- Dateityp: Verzeichnis, normale Datei, Spezialdatei (z.B. Gerät)
- Eigentümer und Gruppe
- Zugriffsrechte
- Zugriffszeiten: letzte Änderung (*mtime*), letzter Zugriff (*atime*), letzte Änderung des *Inodes* (*ctime*)
- Anzahl der *Hard Links* auf den *Inode*
- Dateigröße (in Bytes)
- Adressen der Datenblöcke des Datei- oder Verzeichnisinhalts (zwölf direkte Adressen und drei indirekte)

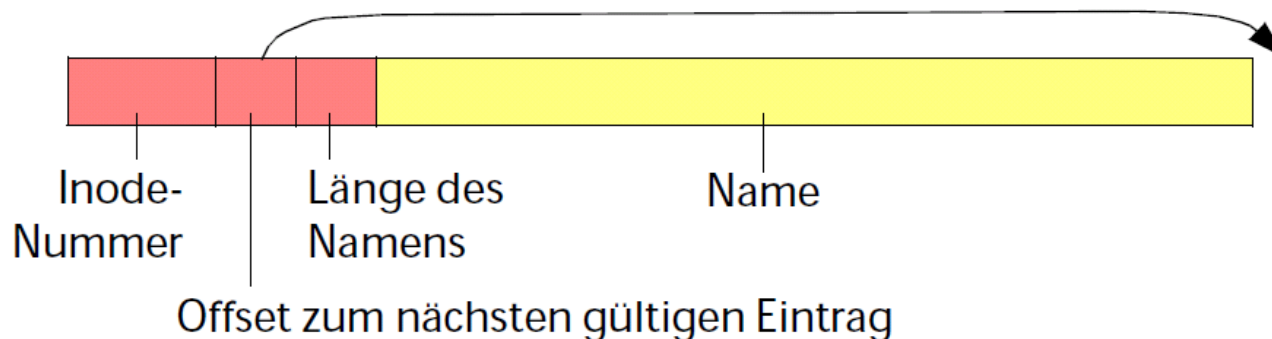
Inodes (4)

Speicherung der Verzeichniseinträge in einer speziellen Datei

- Verzeichniseinträge gleicher Länge
 - z.B. UNIX System V.3

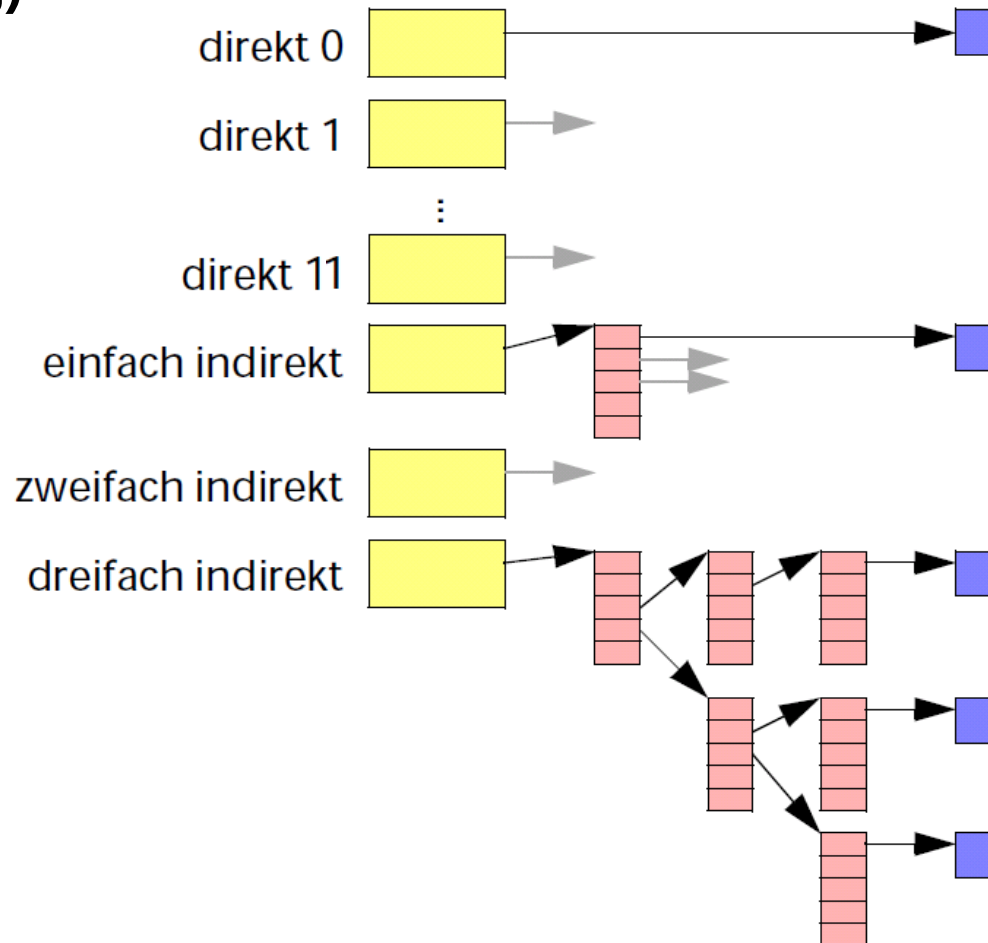


- Verzeichniseinträge variabler Länge
 - z.B. BSD 4.2, System V.4, u.a.



Inodes (5)

Adressierung der Datenblöcke innerhalb des *Inodes* (indizierte Speicherung)



Inodes (6)

Einsatz mehrerer Stufen der Indizierung

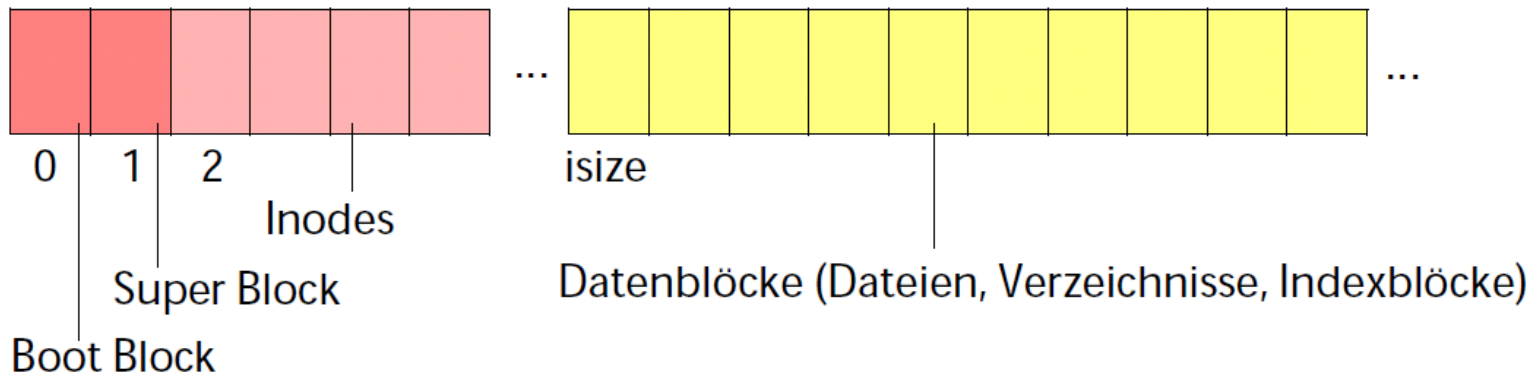
- *Inode* benötigt sowieso einen Block auf der Platte (Verschnitt unproblematisch bei kleinen Dateien)
- Durch mehrere Stufen der Indizierung auch große Dateien adressierbar
- Schnelle Positionierung des Schreib-/Lesezeigers

Nachteil

- Mehrere Blöcke müssen geladen werden (nur bei langen Dateien)

System V File System / UFS (1)

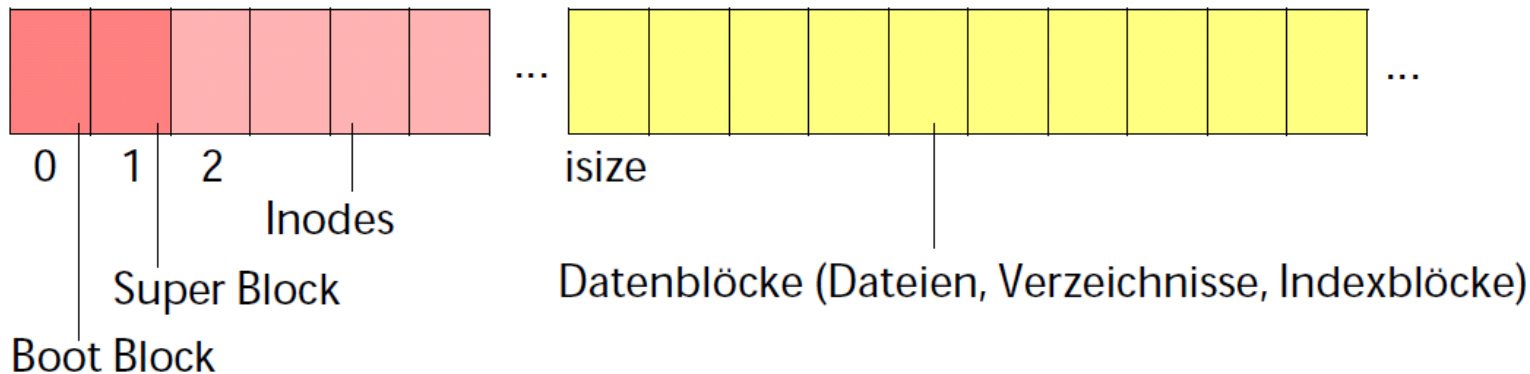
Blockorganisation



- *Boot Block* enthält Informationen zum Laden eines initialen Programms

System V File System / UFS (2)

Blockorganisation



- *Super Block* enthält Verwaltungsinformation für ein Dateisystem
 - Anzahl der Blöcke, Anzahl der *Inodes*
 - Anzahl und Liste freier Blöcke und freier *Inodes*
 - Filesystem-Attribute (z.B. *clean*, *active*)
 - Filesystem-*Label* und Pfadname des letzten *Mount Points*

Roter Faden

5. Filesysteme

- Einleitung
- Beispiel: UNIX/Linux
 - Baumstruktur, Pfade
 - *Links*
 - *Mounten* von Filesystemen
 - *Inodes*
 - UNIX-Filesysteme: UFS, BSD 4.2, EXT2
- Zuverlässige Filesysteme
- Limitierung der Plattennutzung, fehlerhafte Blöcke

Limitierung der Plattennutzung

Mehrbenutzersysteme

- Einzelnen Benutzern sollen verschieden große Kontingente zur Verfügung stehen
- Gegenseitige Beeinflussung soll vermieden werden (*Disk full* Fehlermeldung)

Fehlerhafte Plattenblöcke

Blöcke, die beim Lesen Fehlermeldungen erzeugen

- z.B. Prüfsummenfehler

Hardwarelösung

- Platte und Plattencontroller bemerken selbst fehlerhafte Blöcke und maskieren diese aus
- Zugriff auf den Block wird vom Controller automatisch auf einen „gesunden“ Block umgeleitet

Softwarelösung

- Filesystem bemerkt fehlerhafte Blöcke und markiert diese auch als belegt

Zusammenfassung (1)

Einleitung

- Festplatten: Platten, Sektoren, Spuren, Zylinder
- Bestandteile von Filesystemen: Dateien, Verzeichnisse, Partitionen
- Attribute: Name, Typ, Größe, Zeitstempel, ...
- Operationen: lesen, schreiben, ausführen, erzeugen, löschen, ...

UNIX/Linux

- Baumförmige, hierarchische Organisation
- Harte und symbolische *Links*: „Abkürzungen“ im Verzeichnisbaum
- *Inodes*: zentrale Datenstruktur zur Verwaltung von Dateien und Verzeichnissen; enthalten Dateityp, Größe, Eigentümer, Rechte, ...; 12 direkte, 3 ein- bis dreifach-indirekte Adressen auf Datenblöcke (zumindest in EXT2)
- BSD: Zylindergruppen; EXT2: Blockgruppen